Test 3 review day

CMSE 381

Prof. Mengsen Zhang

Michigan State University
::
Dept of Computational Mathematics, Science & Engineering

Fri, Nov 21, 2025

Announcements

Announcements:

- midterm # 3 on Monday
- Bonus code portfolio assignment due Nov 26 on D2L.
- Final Project submission on D2L. Due Dec 5.

Based on the questions submitted

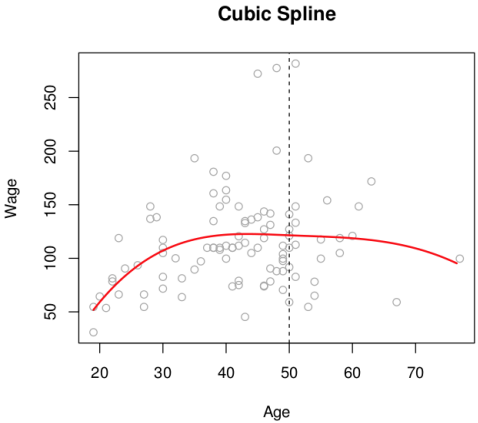
- Cubic spline math
- Bagging and random forests
- SVM kernels
- Convolutional and softmax output

Section 1

Cubic Splines

Cubic splines

- Fit piecewise polynomial
- Require continuity at knots
- Require the first and second derivatives to be continuous at knots
- Now we have smooth joins
- This is called a "cubic spline"
- 8 original degrees of freedom, plus three constraints (continuous, 1st deriv continuous, 2nd deriv continuous) leaves 5 degrees of freedom
- cubic spline with K knots uses $4 + K$ degrees of freedom



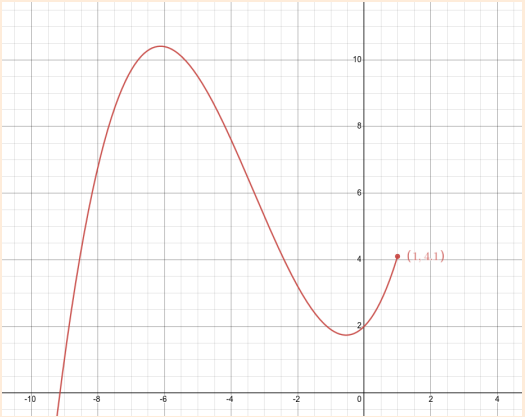
Test your understanding: PollEv

Example

We have the following piecewise cubic polynomial:

$$f(x) = \begin{cases} 2 + x + x^2 + 0.1x^3 & x \leq 1 \\ b_0 + b_1 x + b_2 x^2 - x^3 & x > 1 \end{cases}$$

What are b_1, b_1, and b_2 to make this a cubic spline?



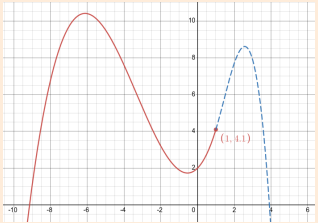
Check your answers: www.desmos.com/calculator/kbm0zivqco

More space for work

$$f(x) = \begin{cases} 2 + x + x^2 + 0.1x^3 & x \leq 1 \\ b_0 + b_1 x + b_2 x^2 - x^3 & x > 1 \end{cases}$$

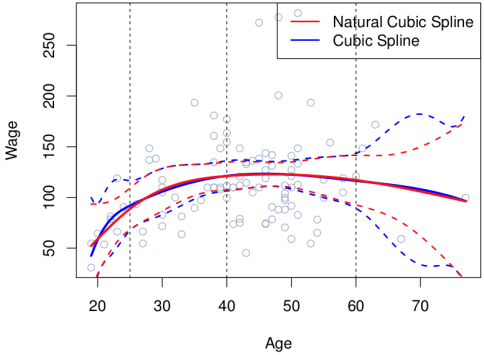
Instructor tool for generating solution while messing with b_3: `https://www.desmos.com/calculator/5prmps9cuo`

$$f(x) = \begin{cases} 2 + x + x^2 + 0.1x^3 & x \leq 1 \\ 3.1 + -2.3x + 4.3x^2 - x^3 & x > 1 \end{cases}$$



Notes on cubic splines

- Human eyes can't detect the discontinuities so often cubic splines are a good way to go
- Still have problem of high variance at the outer range of predictors (big or small X)
- *Natural spline:* regression spline plus boundary constraints where function is required to be linear at the boundary



Section 2

Bagging and Random Forests

Use ensemble of trees to reduce variance

Want to do (but can't):
Build separate models from independent training sets, and average resulting predictions:

- $\hat{f}^1(x), \cdots, \hat{f}^B(x)$ for B separate training sets
- Return the average

$$\hat{f}_{avg}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^b(x)$$

- Decreases variance, but not practical

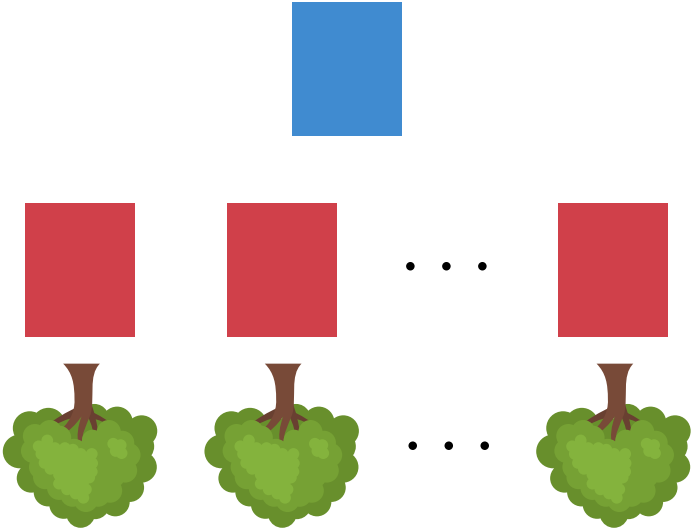
Boostrap modification:

- Work with fixed data set
- Take B samples from this data set (with replacement)
- Train method on bth sample to get $\hat{f}^{*b}(x)$
- Return average of predictions (regression)

$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x)$$

or majority vote (classification)

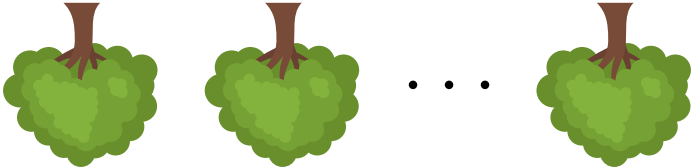
Tree version



- *Blue is original data set* $X : n \times p$
- *Red are the subsampled data sets*
- *Build a tree from each*
- *Get prediction from each, return the average*

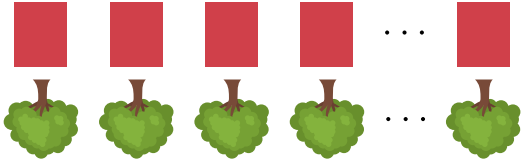
Prediction on new data point

New data point goes in, run through each tree, average outputs



Out of Bag Error Estimation

- On average, bootstrap sample uses about 2/3 of the data *We did this homework problem a while ago I think? Ch 5, Ex. 2*
- Remaining observations not used are called *out-of-bag* (OOB) observations
- For each observation, run through all the trees where it wasn't used for building
- Return the average (or majority vote) of those as test prediction! rather law of large number
- Bootstraped version of LOOCV.



Random Forests

- Goal is to decorrelate the bagged trees:
  - If there is a strong predictor, the first split of most trees will be the same
  - Most or all trees will be highly correlated
  - Averaging highly correlated quantities doesn't decrease variance as much as uncorrelated

- The random forest fix:
  - Each time a split is considered, only use a random subset of m the predictors
  - Fresh sample taken every time
  - Typically $m \approx \sqrt{p}$
  - On average, $(p - m)/p$ of splits won't consider strong predictor
  - $m = p$ gives back bagging

Section 3

SVM Kernels

The kernel - replacing global coordinate with local similarity

$$K(x_i, x_i')$$

- $K(x_i, x_i')$ can be thought of as similarity function.
- An example is inner product $\langle x, x_i \rangle$ will be talked about in more detail on the next slide.
- $\langle x, x_i \rangle$ is known as a linear kernel

$$y_i f(x_i) \geq M(1 - \epsilon_i), \text{ where}$$

$$f(x) = \beta_0 + \sum_{j \in \mathcal{S}} \alpha_i K(x, x_j)$$

- The function defined is as above with whatever choice of K
- When the support vector classifier is combined with a non-linear kernel the resulting classifier is known as a support vector machine.

SVC via inner products

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i \langle x, x_i \rangle$$

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle$$

- Via some magic, there are coefficients α_i which give the linear support vector classifier
- In this notation, the x_i's are all the training points

- How to actually get it is outside the scope of this class
- To estimate the paramters $\alpha_1, \cdots, \alpha_n$ and β_0, need $\binom{n}{2} = n(n-1)/2$ inner products $\langle x_i, x_{i'} \rangle$
- BUT, α is only non-zero for support vectors. so actually few terms

The kernel - more generally

$$K(x_i, x_i')$$

- Swap out my inner product $\langle x, x_i \rangle$ for something potentially more complicated
- $\langle x, x_i \rangle$ is known as a linear kernel

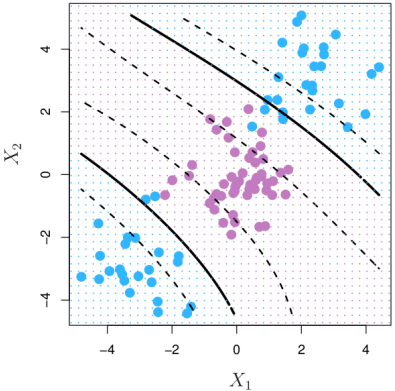
$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i)$$

- The function defined is as above with whatever choice of K
- $K(x_i, x_i')$ can be thought of as similarity function.
- When the support vector classifier is combined with a non-linear kernel the resulting classifier is known as a support vector machine.

A polynomial kernel

$$K(x_i, x_{i'}) = \left(1 + \sum_{j=1}^{p} x_{ij} x_{i'j}\right)^d$$

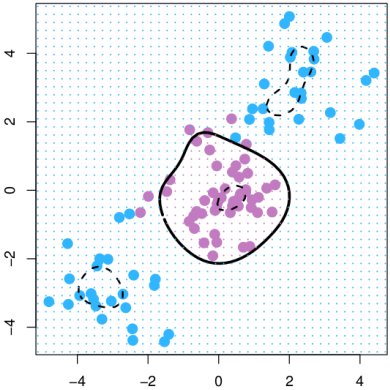
- much more flexible decision boundary.
- amounts to fitting a support vector classifier in a higher-dimensional space involving polynomials of degree d, rather than in the original feature space.
- Right: degree 3 polynomial kernel



A radial kernel

$$K(x_i, x_i') = \exp\left(-\gamma \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2\right)$$

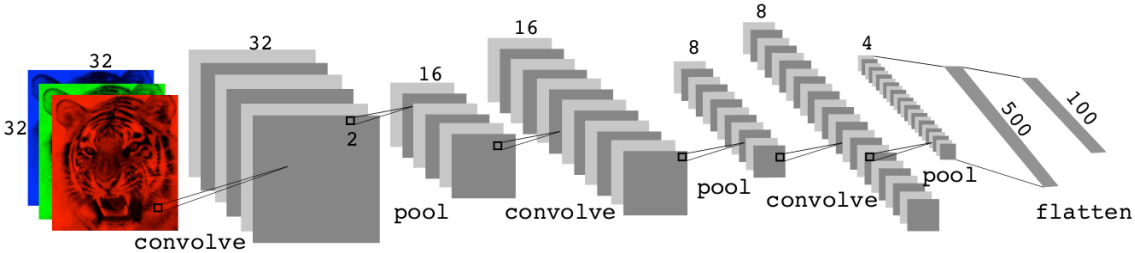
- this is essentially a gaussian kernel
- big distance makes $\sum (x_{ij} - x_{i'j})^2$ big, but then e^{-big} is tiny.
- In $f(x)$, this means that x_i plays little role if it's far away.
- Radial kernel has local behavior



Section 4

Convolutional Neural Networks and softmax output

CNN



- Start with three channels
- conv layer – extract spatial features via convolution with learned filters
- Next is max pool layer, spatial coarse-graining
- convolve & pool several more times
- Since size of image decreased, we increase number of filters in the next convolve layer to compensate
- Flattened, then fed into 1 or more fully-connected layers
- Last is softmax activation for the 100 classes

https://poloclub.github.io/cnn-explainer/

Convolution layer

Convolution Filter

Original Image:

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \\ j & k & l \end{bmatrix}$$

Convolution filter:

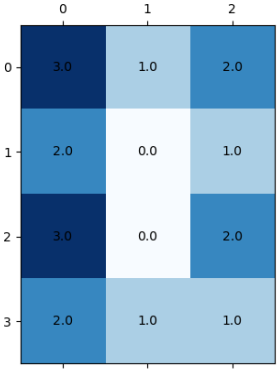
$$\begin{bmatrix} \alpha & \beta \\ \gamma & \delta \end{bmatrix}$$

Convolved Image

$$\begin{bmatrix} a\alpha + b\beta + d\gamma + e\delta & b\alpha + c\beta + e\gamma + f\delta \\ d\alpha + e\beta + g\gamma + h\delta & e\alpha + f\beta + h\gamma + i\delta \\ g\alpha + h\beta + j\gamma + k\delta & h\alpha + i\beta + k\gamma + l\delta \end{bmatrix}$$

Note we are just doing inner product here, which measure(ish) similar

Convolution Filter Example

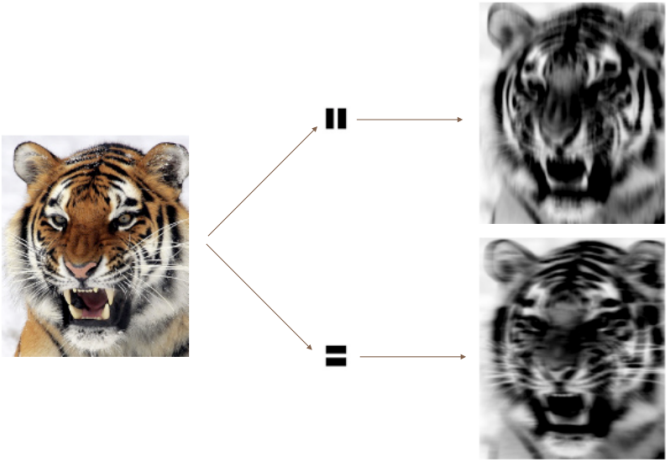


Filter:

$$\begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$$

- $\begin{bmatrix} 1 & 3 \\ 0 & 3 \\ 1 & 3 \end{bmatrix}$
- Convolved image highlights regions that resemble the convolution filter.
- Conv or filtering is traditional signal processing, but in CNN the filter is learned (mentioned again later)
- Note that the result is smaller than original

Convolution filter: Bigger example



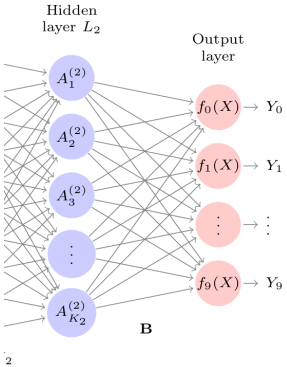
- 192x179 image of tiger
- Convolution filters are 15x15 images with mostly zero (black) narrow strip of ones (white)
- Highlights areas of the tiger that resemble the filter

Pooling layers

- Condense a large image into a smaller summary image
- Lots of options, but max pooling is common:
- For each 2x2 non-overlapping block of pixels, get a single new pixel with the maximum value from the block
- Provides location invariance: so long as large value in one of the four pixels, the whole block registers as a large value

$$\text{Max pool} \begin{bmatrix} 1 & 2 & 5 & 3 \\ 3 & 0 & 1 & 2 \\ 2 & 1 & 3 & 4 \\ 1 & 1 & 2 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix}$$

The last column for classification: Softmax



$$f_m(X) = \Pr(Y = m|X) = \frac{e^{Z_m}}{\sum_{\ell=0}^{9} e^{Z_\ell}},$$

- Want this to act like a probability.
- Answer is to use the softmax activation function $f_m(X)$
- Values are non-negative
- Values sum to 1
- Return class with highest probability